

different distributions learned. The watermark embedding is training a model using specific inputs and outputs to make the model have the behavior (having specific outputs given specific inputs) showing the watermark. First, to merge the distributions of the normal tasks and the watermark, we apply the soft nearest neighbor loss [3] to the training process. Then we find that the watermark may harm the model's performance. Therefore, we propose a method to optimize the watermark embedding so it does less harm to the model performance.

We evaluate the approach on two datasets MSRC-9 and ENZYMES. We demonstrate that our approach can add a watermark resistant to the model extraction attacks to the graph neural networks on cloud services. We further showed that our optimization method can improve the model performance through evaluations.

To summarize, the main contributions of this paper are three-fold:

- We propose a method to add a model-extraction-resistant watermark to the graph neural networks on cloud services.
- We propose an optimization-based technique to improve the performance of the watermarked graph neural networks.
- We perform evaluations to show the effectiveness of our proposed methods.

II. PRELIMINARIES

A. Graph Neural Network

Graph structures commonly exist in many real-world data such as social networks, wireless networks, and financial networks. Traditional machine learning models do not perform well on these data, as they are less effective in making use of network topology. Recently, graph neural networks (GNNs) has emerged as a family of representation learning models that encode the network topology. GNNs adopt the deep learning model framework and learn network topology by aggregating information from nodes' neighbors using methods like network propagation.

GNN can be used for node-level tasks, edge-level tasks, and graph-level tasks. For node-level and edge-level tasks, we can predict some properties of nodes in a graph and edges in a graph, respectively. For the graph-level tasks, we can predict some properties of a whole graph, instead of some inner elements of a graph. In this paper, we focus on graph-level tasks, and the graph data we use are undirected. According to the ways of graph propagation, there are many GNN models, such as GCN, GAT, and GraphSage. In this paper, we use GraphSage as our target model.

B. MLaaS and Model Extraction Attack

In MLaaS, model owners put their models on the cloud and open APIs for customers of their service. A customer can submit their graph data to the model on the cloud and obtain classification results from the model. The classification process of the model happens on the cloud.

Model extraction attack is becoming an appealing attack method with the popularity of deep learning models. In the

model extraction attack, the model that the attacker wants to steal is called *victim model*. The goal of the model extraction attack is to recover a similar or even the same model as the victim model. For an attacker whose target is MLaaS, the only ability they need is to query the model on the cloud. The rationale behind the model extraction attack is to catch the process that converts input data to output. A typical attack is as follows. To launch the model extraction attack, the attacker has dataset $\mathcal{D}'$ analogous to the training dataset $\mathcal{D}$ of the victim model and uses them to query the victim model. The victim model returns the predicted labels. The attacker then trains a model using $\mathcal{D}'$ and the returned labels to mimic the behavior of the victim model. The model trained by the attacker is called *extracted model*. We denote the victim model and extracted model as M and M_e , respectively. For a dataset G , the objective of the model extraction attack is to minimize $\sum_{x \in G} |M(x) - M_e(x)|$. The extracted model has very similar behavior to the victim model if the extraction attack is successful.

III. METHODOLOGY

A. Method Overview

Considering that preventing a stealing action is difficult, we choose to use the watermark to show the model **ownership** with which the model owner can rely on the law to protect her intellectual property. The rationale behind the watermark technique is to hide information about ownership as metadata within the digital property. The hidden information is only held by the watermarked object and is only known by the owner. When the owner wants to verify the ownership of an object, she tries to reveal the information from the object, and show that the information agrees with the special knowledge she holds. For graph neural networks, the information is the watermarked models' special behaviors that are different from the non-watermarked models. More specifically, the special behaviors are the models' certain outputs given certain inputs. The certain inputs and outputs that the model owner wants the watermarked model to hold are called the key inputs and the key labels of the watermark. The set of key inputs and the key labels are denoted as $\{x_i^k\}$ and $\{l_i^k\}$. The i th key label l_i^k corresponds to the i th key input x_i^k .

The watermark framework includes three phases:

- **Key Inputs and Labels Generation.** The owner first generates several key input-label pairs (x_i^k, l_i^k) . The pairs (x_i^k, l_i^k) are outliers only known by the owner. If (x_i^k, l_i^k) is not an outlier, the non-watermarked model will output l_i^k given x_i^k ; therefore, the model's behavior is not special knowledge and can be known by anyone who has a non-watermarked model. In this case, the owner fails to verify her ownership.
- **Watermark Embedding.** The watermark embedding process is training the model with x_i^k and l_i^k . The intuition here is to make the model learn the distribution of the key inputs and labels. If the distribution of the key inputs and labels is learned successfully, M will have output l_i^k given x_i^k .